What are Events in Drupal

Events are just like hooks that tell Drupal to call your function if something happens. We can say Events are Object Oriented Hook System. Drupal Events allow various system components to interact and communicate with one another independently or in a decoupled manner.

Characteristics

Events are part of Drupal's broader adoption of the Symfony framework.
Events are dispatched by certain actions or triggers within the system. You can dispatch events while writing custom code in order to notify other components in the system about actions taken by your code.
Developers can subscribe to these events and define custom actions to be executed when the event occurs.

Example

Drupal core event: KernelEvents::REQUEST
Scenario: Implementing a custom module that listens to the REQUEST event to perform specific actions before the request is processed.

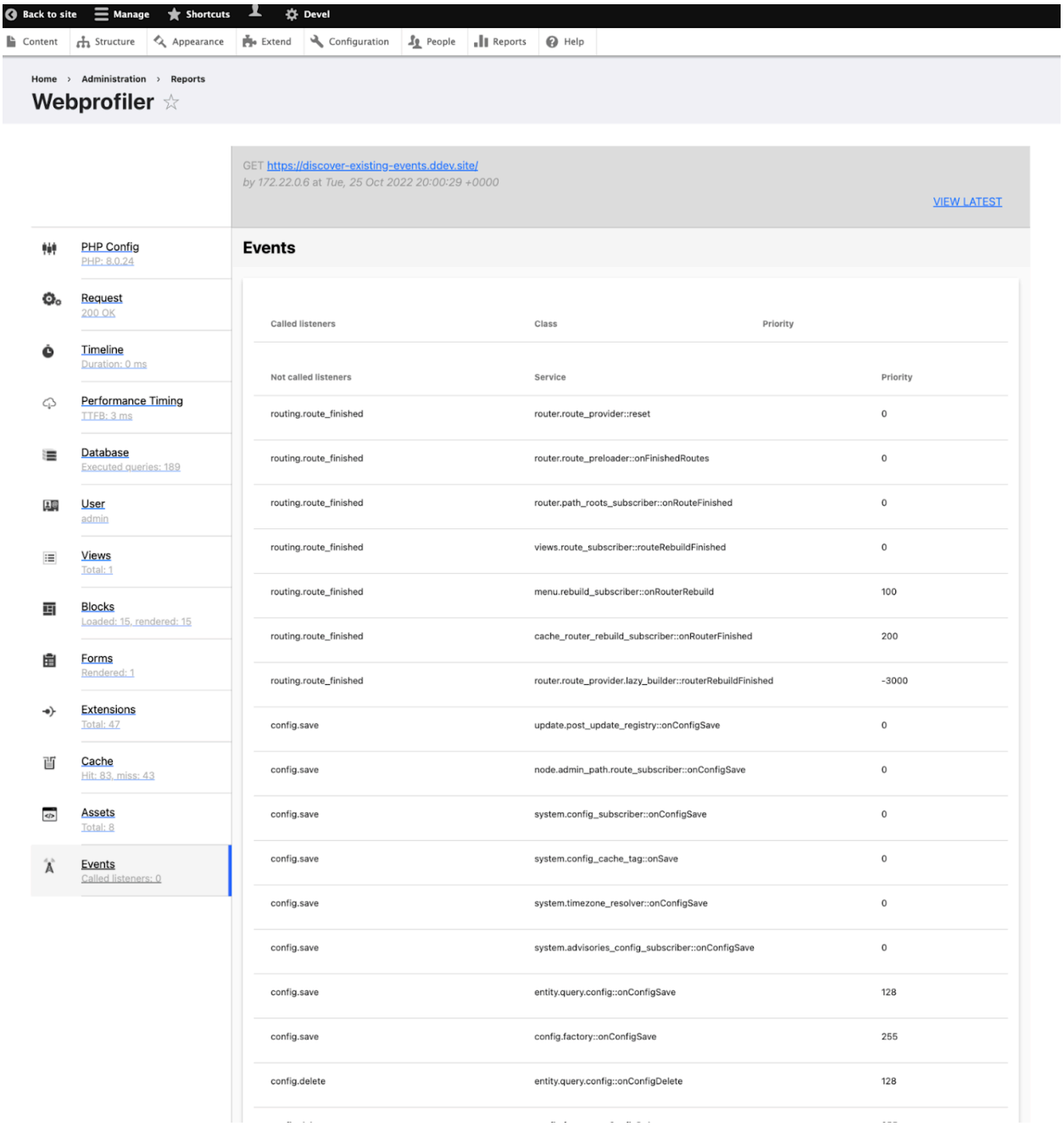
```
// MyModuleEventSubscriber.php
namespace Drupal\my_module\EventSubscriber;

use Symfony\Component\EventDispatcher\EventSubscriberInterface;
use Symfony\Component\HttpKernel\KernelEvents;
use Symfony\Component\HttpKernel\Event\RequestEvent;

class MyModuleEventSubscriber implements EventSubscriberInterface {

  public static function getSubscribedEvents() {
    $events[KernelEvents::REQUEST][] = ['onRequestEvent'];
    return $events;
  }
  public function onRequestEvent(RequestEvent $event) {
    // Custom logic to be executed on every request.
  }
}
```

Discover Existing Events

There are different methods for finding existing events:

1. Using the WebProfiler module:

- Download and enable WebProfiler and Devel module since WebProfiler depends on the Devel module
- Then navigate to Manage > Configuration > Devel Settings > WebProfiler and then select the checkbox to activate the “Events” toolbar item.
- Now while you visit any page on your site you should see the WebProfiler toolbar at the bottom of the page, and after clicking on the events toolbar icon you will get a list of all event subscribers and information that are called during that request.



2. Use Devel to view and event class:

```
drush devel:event
```

```
Enter the number for which you want to get information.:
  [0] kernel.controller
  [1] kernel.exception
  [2] kernel.request
  [3] kernel.response
  [4] kernel.terminate
  [5] kernel.view
> 0

Enter the number to view the implementation.:
  [0] Drupal\path_alias\EventSubscriber\PathAliasSubscriber::onKernelController
  [1] Drupal\Core\EventSubscriber\EarlyRenderingControllerWrapperSubscriber::onController
  [2] Drupal\webprofiler\DataCollector\RequestDataCollector::onKernelController
> 0
```

3. Search in your codebase for @Event:

In your editor such as Visual Studio or PHPStorm, search for text @Event within the file mask: *.php option.

Subscribe to an Event

As we know Drupal uses an event–driven architecture, where various components can communicate with each other by dispatching and subscribing to Events.
Here is an example of subscribing to an Event in Drupal 9/10.
1. Define and Event subscriber service

```
# MyModule/my_module.services.yml
services:
  my_module.event_subscriber:
    class: Drupal\my_module\EventSubscriber\MyModuleEventSubscriber
    tags:
      - { name: event_subscriber }
```

2. Define an Event subscriber class

```
// MyModule/src/EventSubscriber/MyModuleEventSubscriber.php
namespace Drupal\my_module\EventSubscriber;
use Symfony\Component\EventDispatcher\EventSubscriberInterface;
use Symfony\Component\EventDispatcher\Event;

/**
* Class MyModuleEventSubscriber.
*/
```

```
class MyModuleEventSubscriber implements EventSubscriberInterface {

  /**
  * {@inheritdoc}
  */
  public static function getSubscribedEvents() {
    // Specify the event(s) to subscribe to and the method to call when the event occurs.
    $events = [
      'node.insert' => 'onNodeInsert',
      'user.login' => 'onUserLogin',
    ];
    return $events;
  }

  /**
  * React to a node insert event.
  */
  public function onNodeInsert(Event $event) {
    // Your logic here.
    \Drupal::logger('my_module')->notice('Node inserted!');
  }
  /**
  * React to a user login event.
  */
  public function onUserLogin(Event $event) {
    // Your logic here.
    \Drupal::logger('my_module')->notice('User logged in!');
  }
}
```

In this example:

MyModuleEventSubscriber is a class that implements the EventSubscriberInterface.
The getSubscribedEvents method specifies which events the subscriber is interested in and which method to call when each event occurs.
The onNodeInsert and onUserLogin methods contain the logic you want to execute when the corresponding events occur.

Dispatch an Event

In order to allow another developer to subscribe to the Events and react accordingly, you can dispatch an event within your modules or submodules. Before dispatching an Event we need to understand when to

dispatch an event. You can dispatch an event if you want to extend your logic without updating your existing code. Events can be dispatched at any time like creating, updating, loading or deleting data managed by your module.

Let's explain this with an example.

Take a scenario where we want other developers to interact when a new entity (taking node here) is created after submitting your custom form.

1. Create a custom module (if don't have):

```
# Create the module directory
mkdir modules/custom/custom_logger

# Create the module file
touch modules/custom/custom_logger/custom_logger.info.yml
```

2. In custom_logger.info.yml, add the following content:

```
name: 'Custom Logger'
type: module
description: 'Custom module for logging events.'
core_version_requirement: ^8 || ^9 || ^10
package: Custom
```

3. Create an Event:

```
// modules/custom/custom_logger/src/Event/CustomLoggerEvent.php
namespace Drupal\custom_logger\Event;

use Symfony\Component\EventDispatcher\Event;

/**
* Defines the custom event for the custom_logger module.
*/
class CustomLoggerEvent extends Event {

  /**
  * The node that triggered the event.
  *
  * @var \Drupal\node\Entity\Node
  */
```

```
  protected $node;
  /**
  * CustomLoggerEvent constructor.
  *
  * @param \Drupal\node\Entity\Node $node
  *   The node that triggered the event.
  */
  public function __construct($node) {
    $this->node = $node;
  }
  /**
  * Get the node object.
  *
  * @return \Drupal\node\Entity\Node
  *   The node.
  */
  public function getNode() {
    return $this->node;
  }
}
```

4. Dispatch the Event: Creating any entity (taking node here) and dispatching a custom event with a created entity (node) as a parameter.

```
// modules/custom/custom_logger/src/Form/MyCustomForm.php
namespace Drupal\custom_logger\Form;
use Drupal\Core\Form\FormBase;
use Drupal\Core\Form\FormStateInterface;
use Drupal\custom_logger\Event\CustomLoggerEvent;
use Symfony\Component\DependencyInjection\ContainerInterface;
use Symfony\Component\EventDispatcher\EventDispatcherInterface;

/**
* My custom form.
*/
class MyCustomForm extends FormBase {

  /**
  * The event dispatcher service.
  *
  * @var \Symfony\Component\EventDispatcher\EventDispatcherInterface
```

```
 */
 protected $eventDispatcher;

 /**
 * Constructs a new MyCustomForm object.
 *
 * @param \Symfony\Component\EventDispatcher\EventDispatcherInterface $event_dispatcher
 *   The event dispatcher service.
 */
 public function __construct(EventDispatcherInterface $event_dispatcher) {
   $this->eventDispatcher = $event_dispatcher;
 }

 /**
 * {@inheritdoc}
 */
 public static function create(ContainerInterface $container) {
   return new static(
     $container->get('event_dispatcher')
   );
 }

 /**
 * {@inheritdoc}
 */
 public function getFormId() {
   return 'my_custom_form';
 }

 /**
 * {@inheritdoc}
 */
 public function buildForm(array $form, FormStateInterface $form_state) {
   // Build your form elements here.
   return $form;
 }

 /**
 * {@inheritdoc}
 */
```

```
  public function submitForm(array &$form, FormStateInterface $form_state) {
    // Process form submission and create a new node.
    // ...

    // Dispatch the custom event.
    $node = $this->getCreatedNode(); // Implement this method based on your use case.
    $event = new CustomLoggerEvent($node);
    $this->eventDispatcher->dispatch('custom_logger.event', $event);

    // Perform any additional actions after the event is dispatched.
  }
}
```

5. React to Event: Now other modules or parts of the application can now subscribe to the **custom_logger.event** with the created node as a parameter.

What are Hooks in Drupal

Hooks allow modules to alter and extend the existing behavior of Drupal Core or any other module without modifying existing code. Read about update and post update hooks to update your Drupal site in this article.

Characteristics

Drupal's traditional way of allowing modules to interact with the system.
Code can be improved or modified independently and incrementally.
Hooks are predefined functions with specific names that Drupal core or modules call at various points during execution.
Developers implement these functions in their modules to extend or alter default behavior.
Very efficient and easy to implement.

Types of Hooks

- Hooks that react to events: like when the user gets logged in and some action needs to be performed. This is very similar to Events. This is invoked when specific actions are performed. For example **hook_user_cancel()**.
- Hooks that answer questions: like “info hooks”. These are invoked when some component is gathering information about a particular topic. These hooks return arrays whose structure and values are determined in the hook definition. For example, see user module hook user_toolbar() that adds links to the common user account page to the Toolbar.

Note: In Drupal 8 or further versions this is generally handled by the plugin system. Therefore there are very few hooks present now than in Drupal 7.

- Hooks that alter existing data: Alter hooks are generally suffixed with alter. These are invoked to allow modules to alter the existing code. For example: **hook_form_alter()**.

Example:

Drupal core hook: hook_form_alter()
Scenario: Modifying a form defined by another module.

```
// my_module.module
function my_module_form_alter(&$form, \Drupal\Core\Form\FormStateInterface $form_state, $form_id) {	// Custom logic to alter the form.	}
```

Discover existing Hooks

Hooks can be defined by any of contrib or custom modules, even though there are some hooks invoked by Drupal core subsystems like Form API that are always present. This can be a little tricky sometime to find out what hooks are available and which ones to implement.

There are different ways to discover existing hooks:

- Look for the hook definitions in ***.api.php** files contained either in Drupal core or any contributed modules.
- You can use your IDE to search for functions whose name starts with **hook_**.
- You can get a complete list of hooks here.
- Another way is to use drush command that will five you a list of all implementations of a specific hook.

```
drush fn-hook help #Alias of drush devel:hook
```

Invoke a New Hook

- To allow other developers to modify or extends our feature you should invoke a hook or alternatively dispatch an event.
- This can be done on any action like creating, deleting, updating or event when we receive or push some data through API.
- Hooks are invoked using the module_handler **\Drupal::moduleHandler()** services.
- Hooks can be invoked in different ways:

Execute the hook in every module that implements it: **ModuleHandler::invokeAll()**
Execute the hook per-module, usually by looping over a list of enabled modules: **ModuleHandler::invoke()**
Call an alter allowing for alteration of existing data structures using **ModuleHandler::alter()**.

Define a new hook

To define a new hook you should do the following:
1. Choose a unique name for your hook
2. Document your hook

Hooks are documented in a **{MODULE_NAME}.api.php** file:

```
// custom_hooks.api.php
/**
```

```
* Define a custom hook for reacting to specific events.
*
* This hook is invoked when a certain event occurs in the system.
* Modules can implement this hook to perform additional actions in response to the event.
*
* @param string $param1
*   An example parameter for the hook.
* @param array $param2
*   Another example parameter for the hook.
*
* @ingroup custom_hooks_hooks
*/
function hook_custom_event($param1, array $param2) {
  // Your custom hook logic here.
}
```

3. Invoking your hook in your module's code:

```
/**
* Implements hook_ENTITY_TYPE_view().
*/
function hooks_example_node_view(array &$build, EntityInterface $entity,
EntityViewDisplayInterface $display, $view_mode) {
  // Invoke a hook to alert other modules that the count was updated.
  $module_handler = \Drupal::moduleHandler();
  // In this example we're invoking hook_custom_event()
  $module_handler->invokeAll('custom_event', [$entity]);
}
```

When to Use Events or Hooks

Events: Prefer events when actions need to be decoupled or when integrating with Symfony components.
Hooks: Use hooks for more straightforward modifications or when interacting with the Drupal core and contributed modules.

Pros and Cons of using Events VS Hooks

Events:
Pros: Decoupling, better organization, and Symfony integration.
Cons: Slightly steeper learning curve for those unfamiliar with Symfony.
Hooks:
Pros: Simplicity, well-established in Drupal, easier for Drupal – specific tasks.
Cons: Tighter coupling, less organization in larger projects.